

UD3. DISEÑO Y REALIZACIÓN DE PRUEBAS

1.ÍNDICE

1.INTRODUCCIÓN

2.TÉCNICAS DE DISEÑO DE CASOS DE PRUEBA

3. PRUEBAS DE UNIDAD

4. PRUEBAS DE INTEGRACIÓN

5. PRUEBAS DE VALIDACIÓN

6. PRUEBAS DE SISTEMA

7. DOCUMENTACIÓN DE LAS PRUEBAS

8. PLANIFICACIÓN DE LAS PRUEBAS

1.INTRODUCCIÓN

1.INTRODUCCIÓN

Una de las fases del ciclo de vida del software antes de entregar un programa para su explotación es la fase de pruebas. Las pruebas de software consisten en verificar y validar un producto software antes de su puesta en marcha.

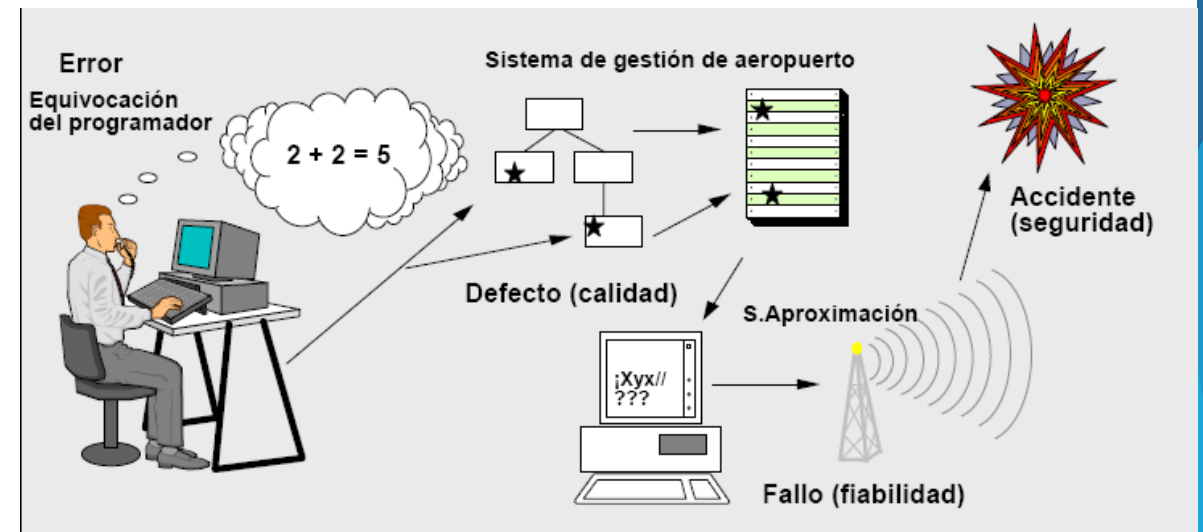
Las pruebas del sistema ayudan a garantizar la calidad de este. Una vez desarrollado el sistema hay que probar si este funciona correctamente y si cumple las especificaciones indicadas por el cliente, por ello, los objetivos de las pruebas son los siguientes:

- ▶ Comprobar que el sistema hace lo que tiene que hacer
- ▶ El sistema realiza sus funciones correctamente
- ▶ Lo hace de una manera eficiente

1.INTRODUCCIÓN

El hecho de que se encuentren errores a lo largo de la fase de pruebas de una aplicación no debe entenderse como un fracaso sino como algo positivo, pues esto permite eliminar los errores antes de entregar la aplicación y evita que estos sean descubiertos por el cliente.

Las pruebas siempre comienzan por los componentes más pequeños. Se empieza probando pequeñas partes del software y se va incrementando de manera progresiva el alcance de la prueba. Se suelen realizar 4 tipos de pruebas secuenciales.



1.INTRODUCCIÓN

Las pruebas del software consisten en verificar y validar un producto software antes de su entrega.

- ▶ Etapas:
- ▶ Planificación de pruebas.
- ▶ Diseño y construcción de los casos de prueba.
- ▶ Definición de los procedimientos de prueba.
- ▶ Ejecución de las pruebas.
- ▶ Registro de resultados obtenidos.
- ▶ Registro de errores encontrados.
- ▶ Depuración de los errores.
- ▶ Informe de los resultados obtenidos.

2.TÉCNICAS DE DISEÑO DE CASOS DE PRUEBA

2. TÉCNICAS DE DISEÑO DE CASOS DE PRUEBA

Para la realización de las pruebas se somete al sistema a una serie de Planes de Prueba con el fin de encontrar todos los errores del sistema, introducir mejoras en el mismo, modificar elementos, etc.

Un **caso de prueba** es un conjunto de entradas, condiciones de ejecución y resultados esperados, desarrollado para conseguir un objetivo particular o condición de prueba. Para llevar a cabo un caso de prueba, es necesario:

- ▶ Definir las **precondiciones** y **postcondiciones**.
- ▶ Identificar unos **valores de entrada**.
- ▶ Conocer el **comportamiento** que debería tener el sistema ante dichos valores.

Tras realizar ese análisis e introducir dichos datos en el sistema, se observa si su comportamiento es el previsto o no y por qué. De esta forma se determina si el sistema ha pasado o no la prueba.

El objetivo principal de los casos de prueba es encontrar el número máximo de errores en el sistema, la ausencia de errores es prácticamente imposible.

2. TÉCNICAS DE DISEÑO DE CASOS DE PRUEBA

Por tanto, las pruebas ayudan a comprobar si el sistema es:

- ▶ **Correcto:** el sistema hace lo que tiene que hacer.
- ▶ **Eficiente:** el sistema hace lo que debe consumiendo el menor número de recursos y consiguiendo tiempos de respuesta lo más cortos posible.
- ▶ **Eficaz:** el sistema debe resolver todos los casos que sea posible de la mejor forma.

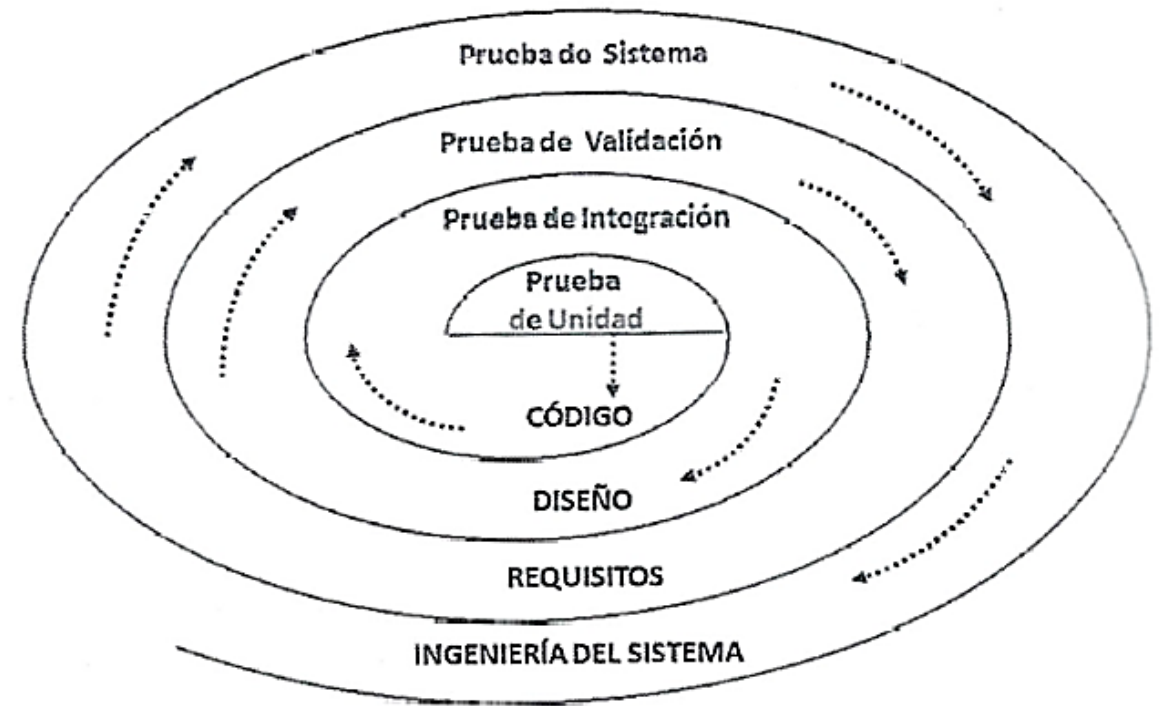
Según lo que tratemos de comprobar, se utilizarán dos técnicas:

- ▶ Técnicas de **Verificación** del sistema: se usan para comprobar si el sistema está bien implementado, es decir, *si ¿El sistema hace lo que debe hacer?*
- ▶ Técnicas de **Validación** del sistema: se usan para comprobar que el sistema realiza lo que el cliente quiere, es decir, *si ¿El sistema hace lo que el cliente quiere?*

2. TÉCNICAS DE DISEÑO DE CASOS DE PRUEBA

Dado que la prueba exhaustiva del sistema es imposible, se utilizarán técnicas que ayuden a la detección del máximo número de errores en el sistema. La estrategia de prueba del software se puede ver como una espiral, en las que vemos los cuatro tipos de pruebas a realizar:

- Pruebas de Unidad:
 - a) Pruebas de Caja Blanca
 - b) Pruebas de Caja Negra
- Pruebas de Integración:
 - a) Pruebas de Integración no Incremental
 - b) Pruebas de Integración Incremental
- Pruebas de Validación
 - a) Prueba Alfa
 - b) Prueba Beta
- Pruebas del sistema:
 - a) Prueba de recuperación
 - b) Prueba de seguridad
 - c) Prueba de resistencia.



2. TÉCNICAS DE DISEÑO DE CASOS DE PRUEBA

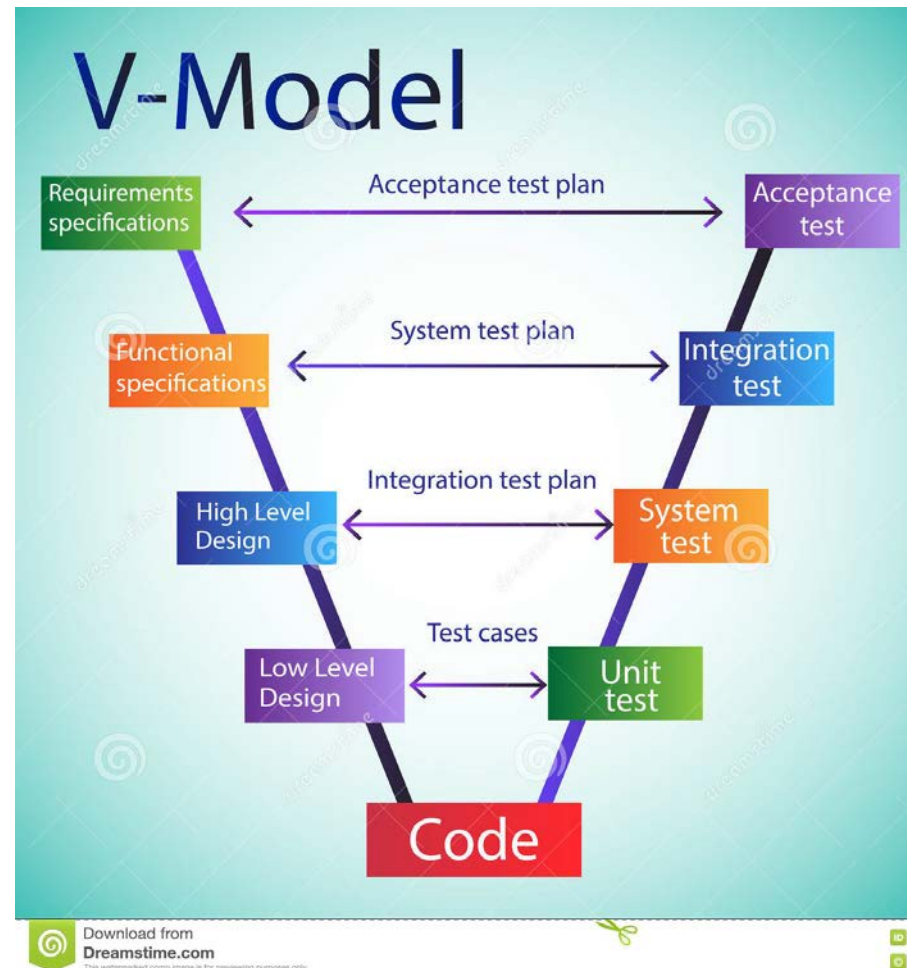
El orden de realización es el siguiente:

- ▶ Las **pruebas de unidad** se centran en la unidad más pequeña del software: el módulo.
- ▶ La **prueba de integración** construye un programa con los módulos probados de forma unitaria y valida la ejecución coordinada de todos ellos. Se valida el diseño.
- ▶ La **prueba de validación** prueba el software en el entorno real de trabajo con intervención del usuario final. Se validan los requisitos de la fase de análisis.
- ▶ La **prueba del sistema** verifica que cada elemento encaja de forma adecuada y se alcanza la funcionalidad y rendimiento total.



2. TÉCNICAS DE DISEÑO DE CASOS DE PRUEBA

Todas estas pruebas se organizan en un modelo equivalente al modelo en cascada que se llama Modelo en V, que se puede representar con el siguiente esquema:



3. PRUEBAS DE UNIDAD

3.1. PRUEBAS DE CAJA BLANCA

3.1.1. PRUEBA DEL CAMINO BÁSICO

3.2. PRUEBAS DE CAJA NEGRA

3.2.1. CLASES DE EQUIVALENCIA (PARTICIÓN)

3.2.1.1. ANÁLISIS DE VALORES LÍMITE

3. PRUEBAS DE UNIDAD

En este nivel del proceso de pruebas se prueba cada unidad del sistema con el fin de eliminar errores en la lógica interna y en la interfaz.

En caso de que se desarrolle software convencional (prog. estructurada) en las pruebas unitarias se prueban todos los módulos de software de forma individual.

Si se trata de software orientado a objetos se probarán las clases de forma individual. En este caso, la prueba de unidad conlleva realizar también pruebas a nivel de método, es decir, para cada método no trivial habrá que comprobar si su comportamiento es el adecuado. Así, para probar correctamente una clase entera, es necesario realizar pruebas con cada uno de sus métodos.

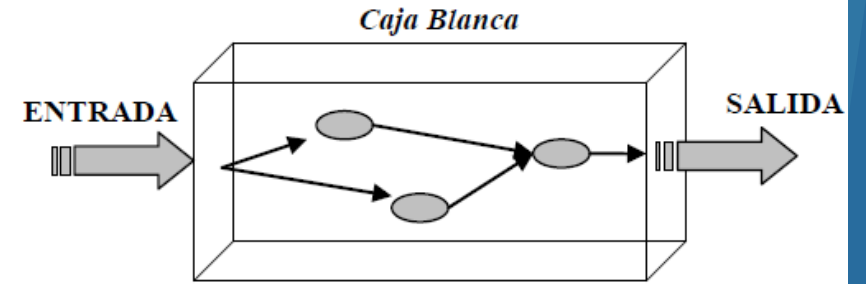
Para ello se suelen utilizar técnicas de **caja blanca** (validan la estructura interna del programa y necesitan conocer los detalles procedimentales del código) y de **caja negra** (necesitan saber la funcionalidad que el código desarrollado ha de proporcionar). Ambos tipos de pruebas no son excluyentes y se pueden combinar para detectar diferentes tipos de errores.

3. PRUEBAS DE UNIDAD

Se realizan pruebas sobre:

- ▶ La interfaz del módulo, para asegurar que los datos fluyen adecuadamente.
- ▶ Las estructuras de los datos locales, para asegurar su integridad durante la ejecución del programa.
- ▶ Las condiciones límite.
- ▶ Todos los caminos de la estructura de control para asegurar que todas las instrucciones se ejecutan al menos una vez.
- ▶ Todos los caminos de manejo de errores.

3.1. PRUEBAS DE CAJA BLANCA



También se las conoce como pruebas estructurales o de caja de cristal.

Se basan en el minucioso examen de los detalles procedimentales del código de la aplicación.

Permiten examinar la estructura interna del programa, es decir, se fijan en el código, si los bucles, las sentencias condicionales, los puntos de toma de decisión, etc. funcionan correctamente, y emiten resultados para todos los caminos que se puedan seguir.

Este tipo de pruebas serán realizadas solamente por los programadores o por personas que conozcan el código, los datos y los flujos.

3.1. PRUEBAS DE CAJA BLANCA

Las pruebas de caja blanca intentan garantizar que:

- ▶ Se ejecutan al menos una vez todos los caminos independientes de cada módulo.
- ▶ Se ejecutan todas las sentencias al menos una vez.
- ▶ Se ejecutan todas las decisiones lógicas en su parte verdadera y en su parte falsa.
- ▶ Se ejecutan todos los bucles en sus límites.
- ▶ Se utilizan todas las estructuras de datos internas para asegurar su validez.

Ejemplo: Prueba del camino básico.

3.1.1.PRUEBA DEL CAMINO BÁSICO

Es una técnica de caja blanca que:

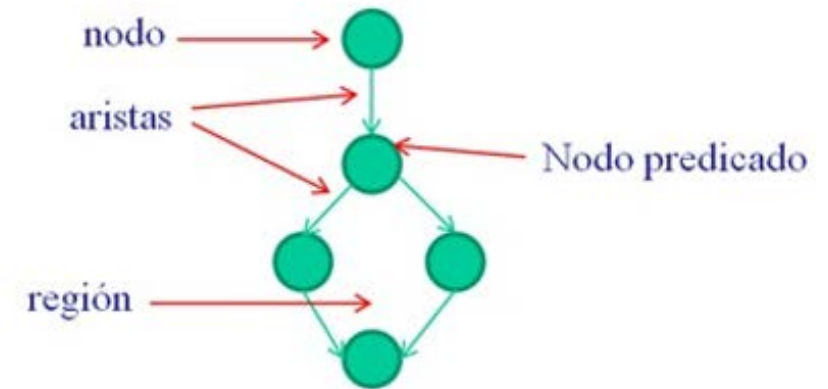
- ▶ Permite que el diseñador de casos de prueba obtenga una medida de la complejidad lógica de un diseño procedimental (***complejidad ciclomática***) y que use esta medida como guía para definir un conjunto de caminos de ejecución.
- ▶ ***Los casos de prueba deben garantizar que se ejecuta cada instrucción del programa por lo menos una vez durante la prueba.***
- ▶ Conceptos:
 - ▶ Complejidad ciclomática
 - ▶ Conjunto de caminos independientes
 - ▶ Casos de prueba

Para la realización de estas pruebas es conveniente construir un **grafo de flujo** de control y sobre él escoger una serie de caminos lógicos que se van a ejecutar usando distintos datos para comprobar el correcto funcionamiento del código.

3.1.1.PRUEBA DEL CAMINO BÁSICO

Grafo de flujo

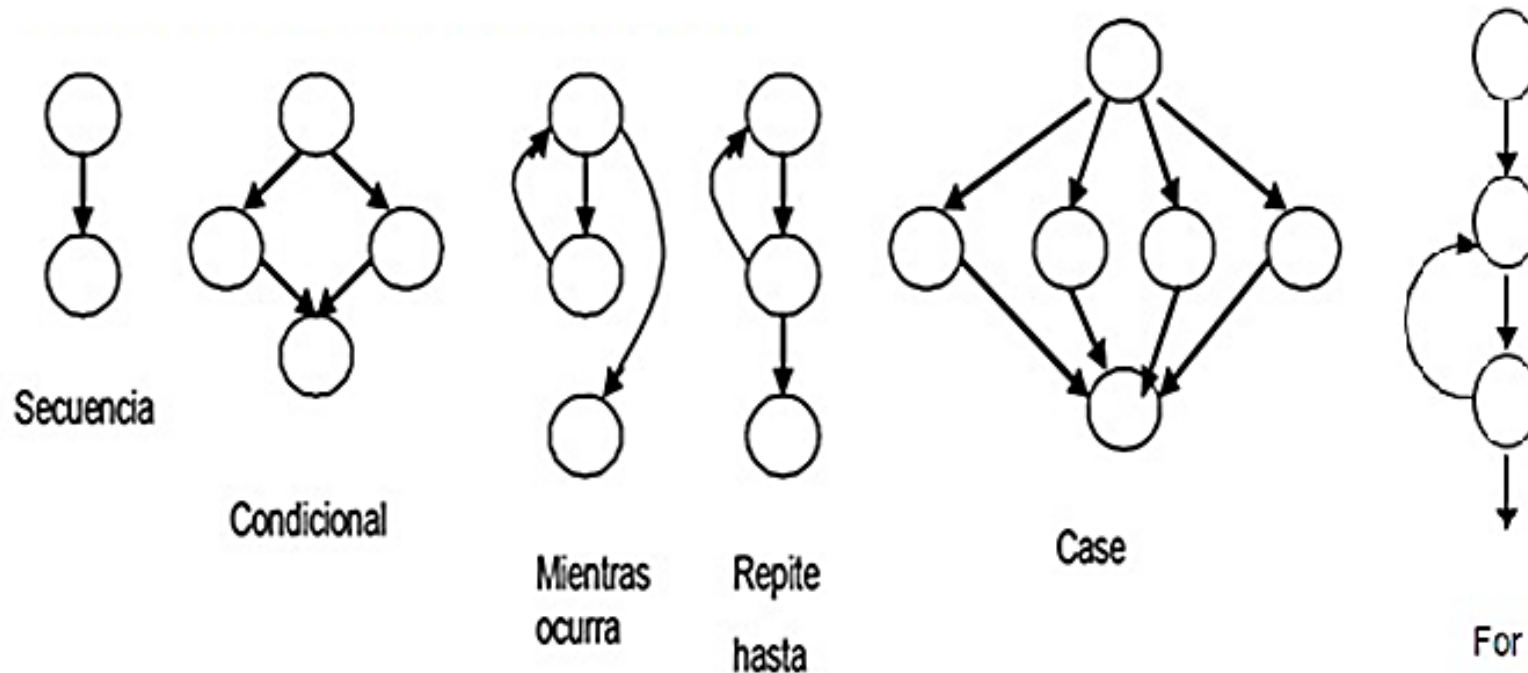
Cualquier representación del diseño procedimental se puede traducir a un grafo de flujo. En un diagrama de flujo se numeran cada uno de los símbolos y los finales de las estructuras de control.



- ▶ Cada **nodo** representa una o más sentencias. También se usan para representat el inicio y el fin del grafo.
- ▶ Las **aristas** o enlaces representan el flujo de control. Una arista termina en un nodo.
- ▶ Las áreas delimitadas por aristas y nodos se llaman **regiones**. El área exterior del grafo constituye otra región adicional.
- ▶ El nodo que contiene una condición se llama **nodo predicado** y se caracteriza porque de él salen dos o más aristas.

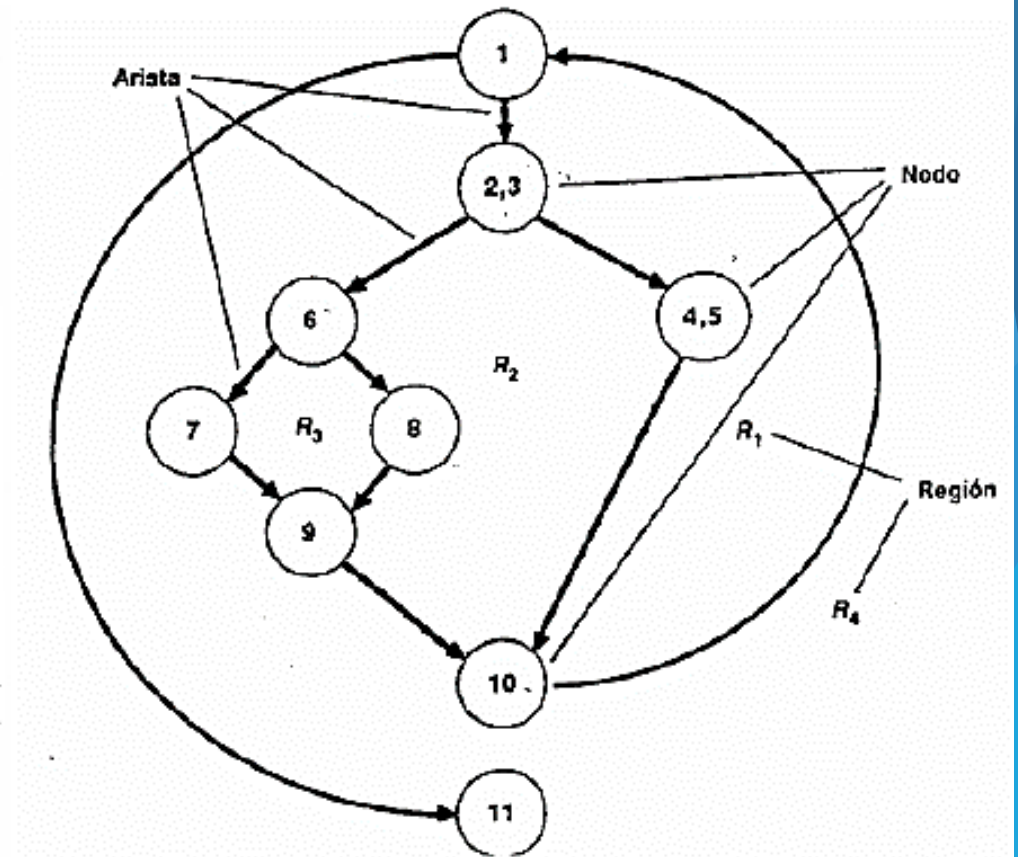
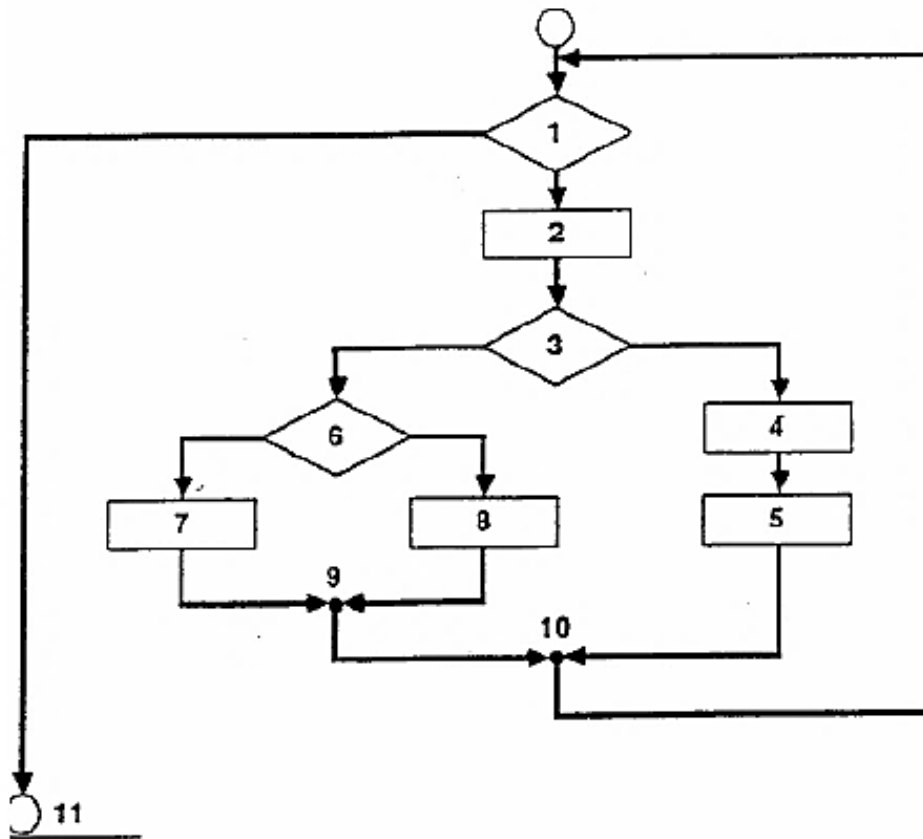
3.1.1.PRUEBA DEL CAMINO BÁSICO

Las estructuras de programación, se pueden representar en los grafos de flujo como sigue:



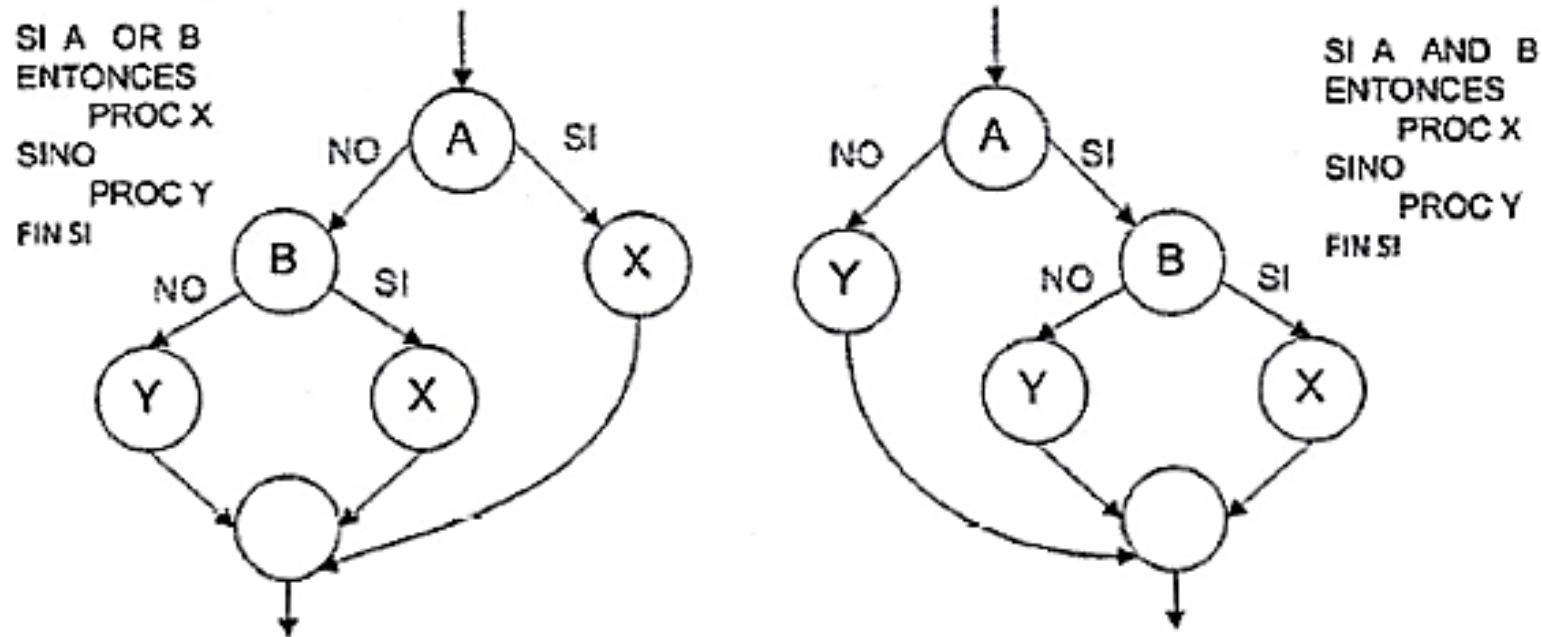
3.1.1.PRUEBA DEL CAMINO BÁSICO

Esto es un **ejemplo** basado en un grafo de flujo que representa la estructura de control del programa representado por el diagrama de flujo de la izquierda.



3.1.1.PRUEBA DEL CAMINO BÁSICO

Si en el diseño procedimental se utilizan **condiciones compuestas**, la generación del grafo de flujo tiene que descomponer las condiciones compuestas en condiciones sencillas, tal y como muestra la figura siguiente.



3.1.1.PRUEBA DEL CAMINO BÁSICO

Complejidad Ciclomática de McCabe

La complejidad ciclomática, $V(G)$, es una métrica del software que proporciona una medida cuantitativa de la complejidad lógica de un programa.

Proporciona el número necesario de casos de prueba que se deben ejecutar para garantizar que cada instrucción del programa se haya ejecutado por lo menos una vez.

La complejidad ciclomática, $V(G)$, puede calcularse de tres formas:

1. $V(G)$ = número de regiones del grafo.
2. $V(G)$ = Aristas - Nodos + 2.
3. $V(G)$ = Nodos predicado + 1.

El valor de $V(G)$ define el número de caminos independientes que existen en un grafo. Un **camino independiente** es cualquier camino del programa que introduce un nuevo conjunto de sentencias o una condición de proceso.

Una vez obtenidos los caminos independientes es preciso construir los casos de prueba que fuerzan la ejecución de cada camino.

3.1.1.PRUEBA DEL CAMINO BÁSICO

Complejidad Ciclomática de McCabe

Como ya hemos dicho, la complejidad ciclomática es una métrica del software que proporciona una medida cuantitativa de la complejidad lógica de un programa. Según el valor obtenido, así de complejo será nuestro programa:

Complejidad Ciclomática	Riesgo
1-10	Programa simple, sin mucho riesgo
11-20	Programa medianamente complejo, riesgo moderado
21-50	Programa complejo, alto riesgo
> 50	Programa no testeable, riesgo muy alto

3.1.1.PRUEBA DEL CAMINO BÁSICO

EJEMPLO

1. $V(G) = \text{aristas} - \text{nodos} + 2$

El grafo tiene 11 aristas y 9 nodos

$$V(G) = 11 - 9 + 2 = 4$$

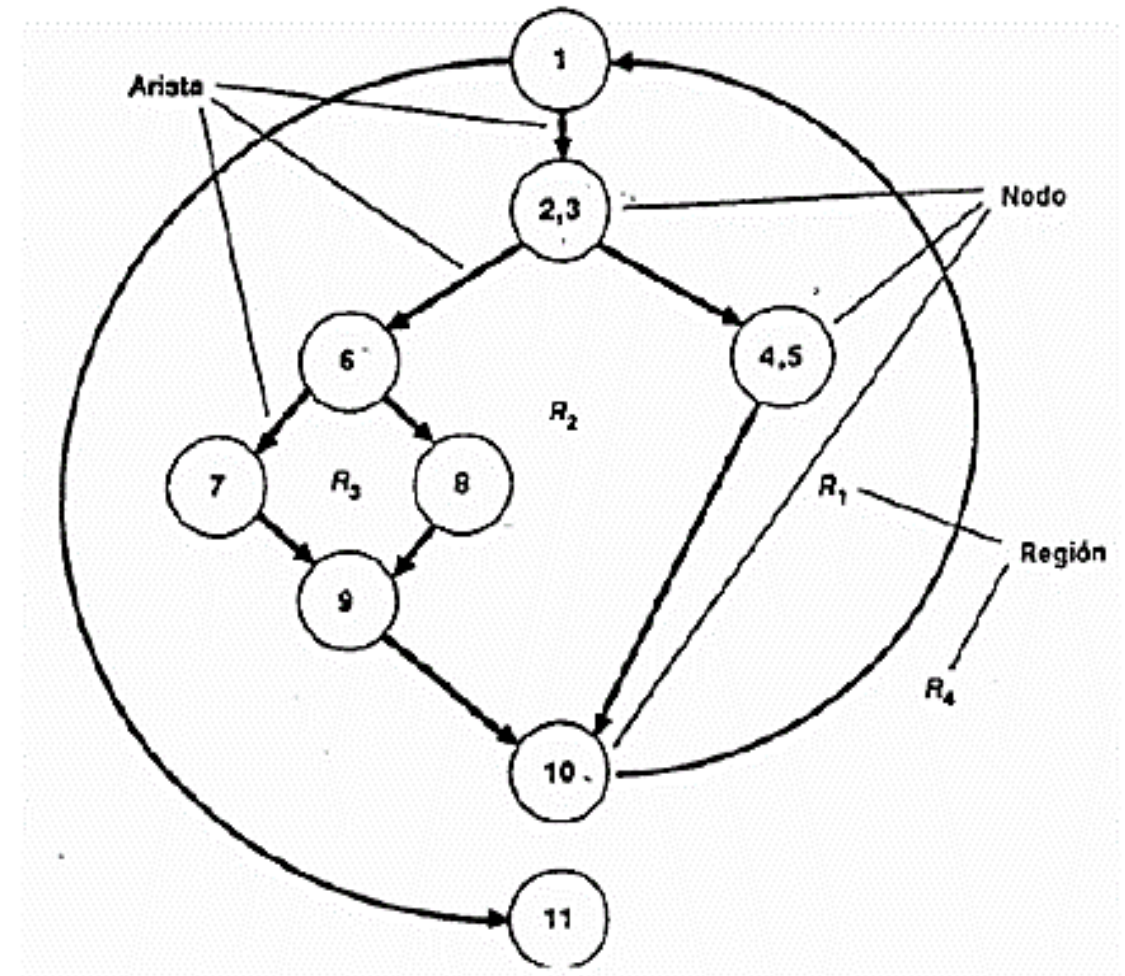
2. $V(G) = \text{regiones}$

El grafo tiene 4 regiones (R_1 , R_2 , R_3 y R_4)

3. $V(G) = \text{nodos predicado} + 1$

El grafo tiene 3 nodos predicado (1, 2+3 y 6)

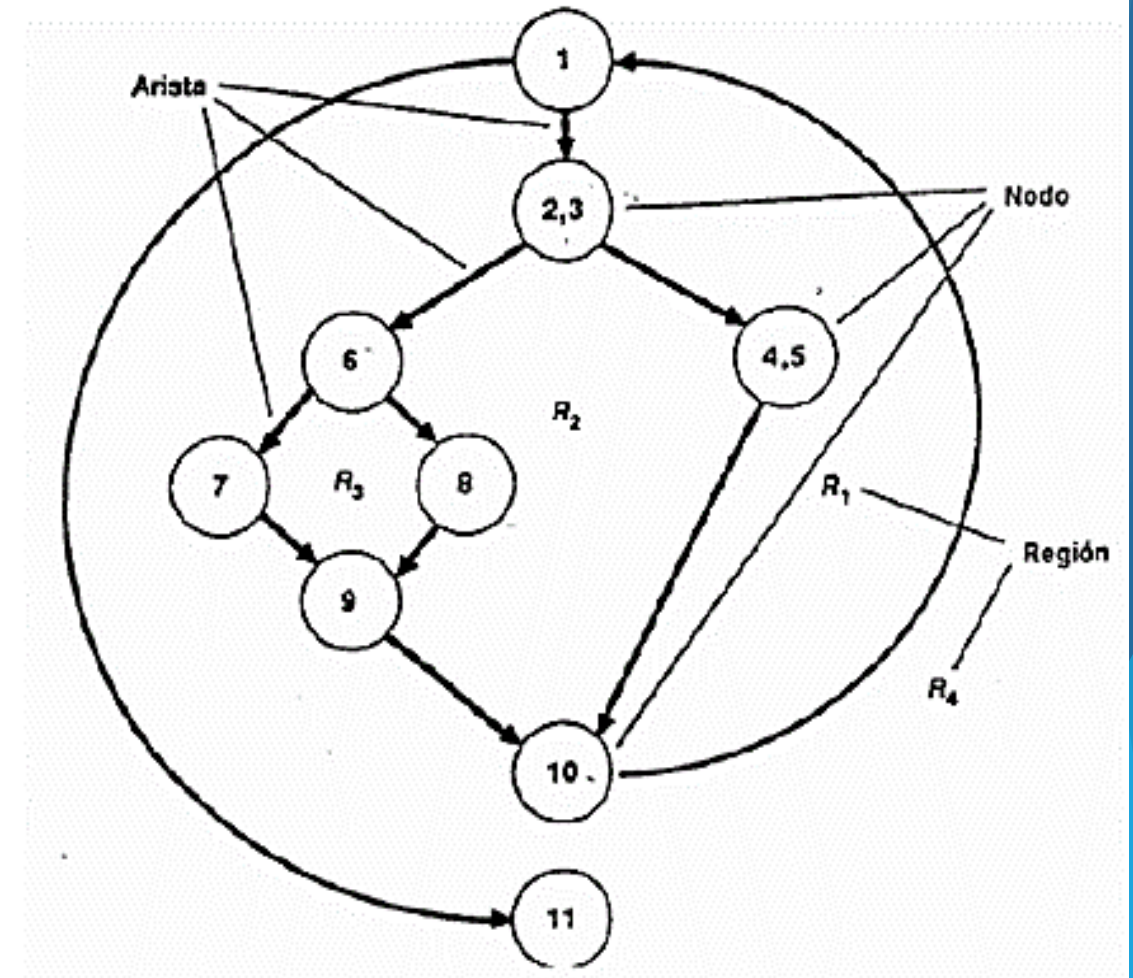
$$V(G) = 3 + 1 = 4$$



3.1.1.PRUEBA DEL CAMINO BÁSICO

Como $V(G)$ es 4, tenemos que buscar los **4 caminos básicos**:

- 1-11
- 1-2-3-4-5-10-1-11
- 1-2-3-6-7-9-10-1-11
- 1-2-3-6-8-9-10-1-11



3.1.1.PRUEBA DEL CAMINO BÁSICO

Inicio

Abrir Fichero Alumnos
Leer Alumnos (Curso, Nombre, Sexo, Nota) (1)

Mientras [Haya registros] **Hacer** (2)

(3) NH = 0, NM = 0
Visualizar Curso (4) (5)

Mientras [Haya registros] y [Mismo curso] **Hacer**

Si [sexo = "H"] (6) Entonces

[NH = NH + 1] (8)

Sino

[NM = NM + 1] (7)

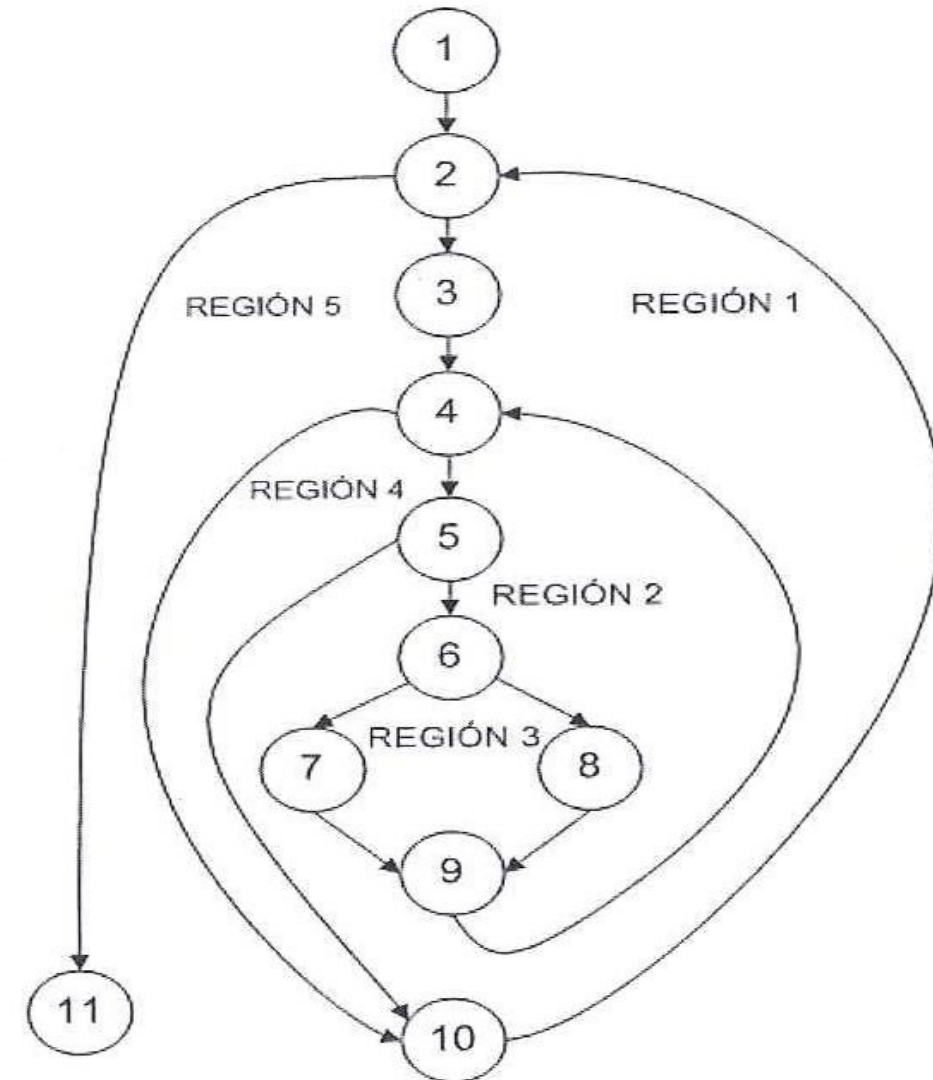
Fin si

Leer Alumnos (Curso, Nombre, Sexo, Nota) (9)

Fin Mientras
Visualizar NH, NM (10)

Fin Mientras
Abrir Fichero Alumnos (11)

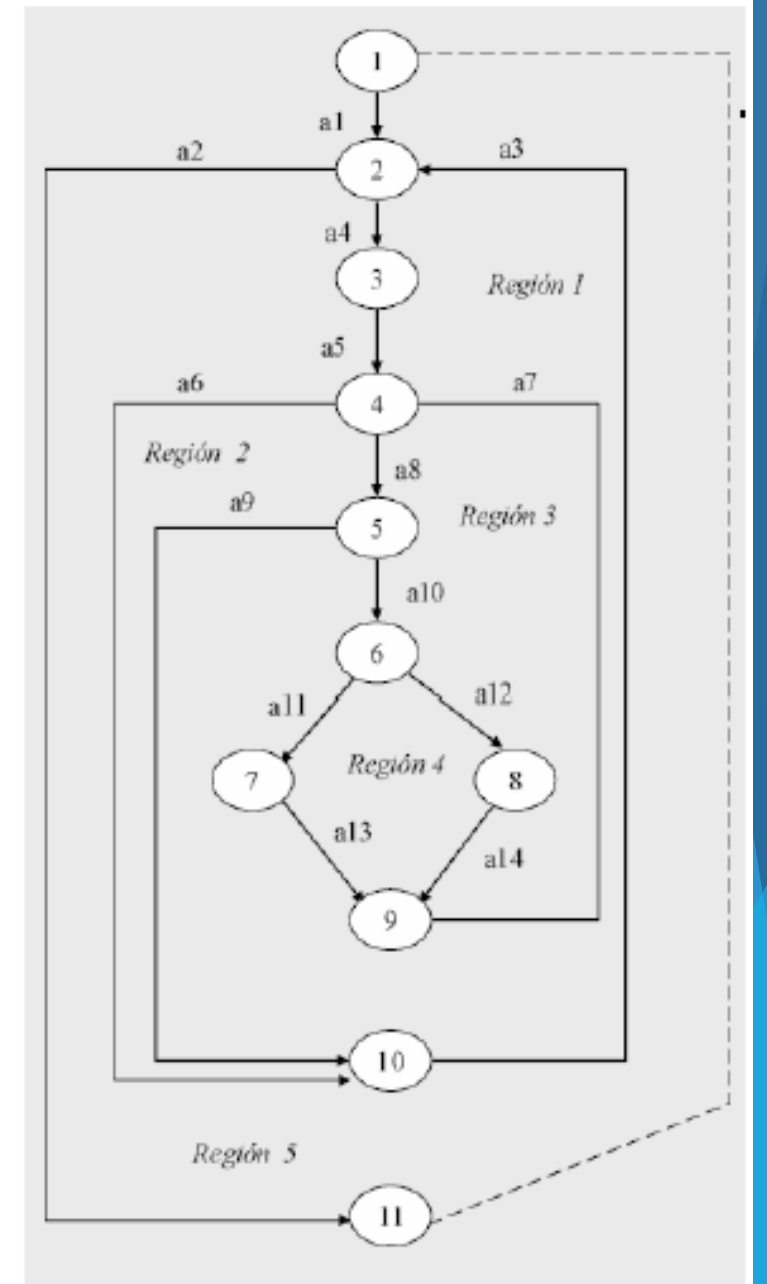
Fin



3.1.1.PRUEBA DEL CAMINO BÁSICO

EJEMPLO

- 1 – 2 – 11
- 1 – 2 – 3 – 4 – 10 – 2
- 1 – 2 – 3 – 4 -- 5 – 10 – 2
- 1 – 2 – 3 – 4 -- 5 – 6 – 7 - 9
- 1 – 2 – 3 – 4 -- 5 – 6 – 8 – 9



3.1.1.PRUEBA DEL CAMINO BÁSICO

PROCEDIMIENTO

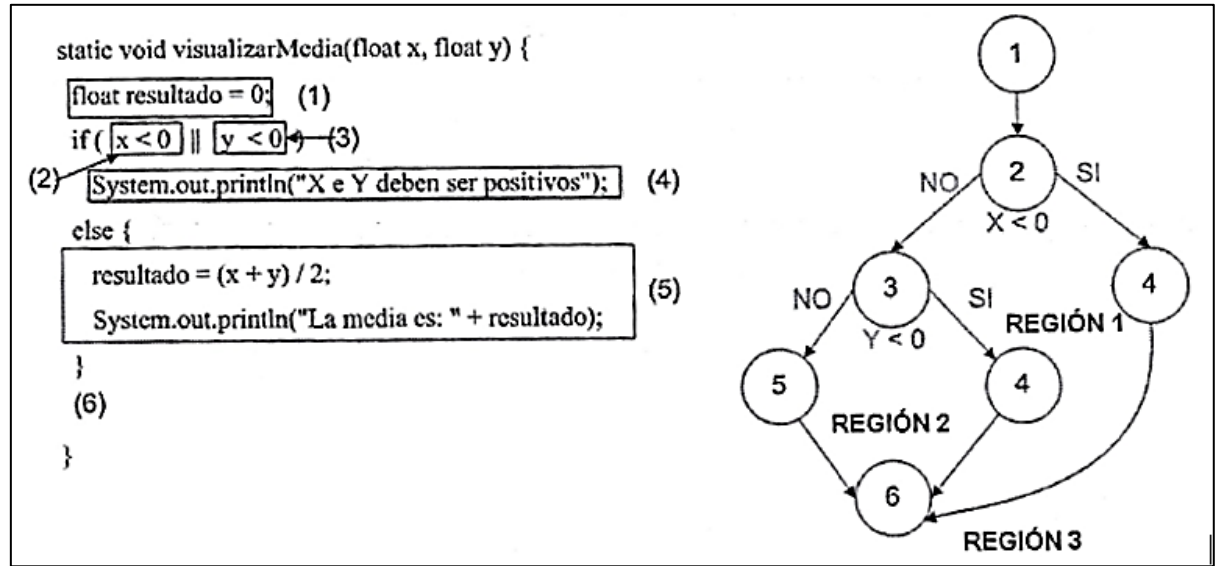
Los pasos que hay que seguir para realizar el diseño de pruebas mediante la técnica del camino básico son:

1. Dibujar el grafo de flujo tomando como base el diseño o el propio código del programa.
2. Determinar la complejidad ciclomática del grafo.
3. Determinar un conjunto de caminos linealmente independientes.
4. Preparar los casos de prueba para cada uno de los caminos del grafo anterior.

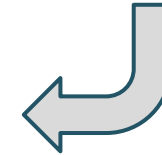
3.1.1.PRUEBA DEL CAMINO BÁSICO

Ejemplo

```
static void visualizarMedia(float x, float y) {  
    float resultado = 0;  
    if (x<0 || y<0)  
        System.out.println("X e Y deben ser positivos");  
    else {  
        resultado = (x+y)/2;  
        System.out.println("La media es: " + resultado);  
    }  
}
```



Camino	Caso de prueba	Resultado esperado
Camino 1: 1-2-3-5-6	Escoger algún X e Y tal que NO se cumpla la condición $X < 0 \parallel Y < 0$ $X=4, Y=5$ <code>visualizarMedia(4,5)</code>	Visualiza: La media es: 4.5
Camino 2: 1-2-4-6	Escoger algún X tal que SÍ se cumpla la condición $X < 0$ (Y puede ser cualquier valor) $X=-4, Y=5$ <code>visualizarMedia(-4,5)</code>	Visualiza: X e Y deben ser positivos
Camino 3: 1-2-3-4-6	Escoger algún X tal que NO cumpla la condición $X < 0$ y escoger algún Y que SÍ cumpla la condición $Y < 0$ $X=4, Y=-5$ <code>visualizarMedia(4,-5)</code>	Visualiza: X e Y deben ser positivos



3.2. PRUEBAS DE CAJA NEGRA



Las pruebas de Caja Negra son aquellas que comprueban el funcionamiento de un componente software a través de su interfaz, es decir, de sus entradas y salidas, sin necesidad de conocer su estructura o funcionamiento interno.

Tratan de comprobar que ante determinadas entradas, se producen las salidas esperadas. El sistema es similar a una caja negra.

También se les suele denominar pruebas de **comportamiento**.

3.2. PRUEBAS DE CAJA NEGRA

Estas pruebas se pueden ver desde un enfoque funcional del sistema, ya que se diseñan los casos de prueba y los datos de prueba a partir de las especificaciones funcionales del sistema, se busca probar completamente:

- ▶ Las funciones realizadas por el sistema.
- ▶ El cumplimiento de los objetivos del sistema.
- ▶ Las reacciones del sistema ante los estímulos exteriores.

Para comprobar el correcto funcionamiento del sistema se introducen:

- ▶ valores límites. (Método de análisis de valores límite)
- ▶ datos erróneos, valores susceptibles a crear problemas.
- ▶ datos equivalentes (Método de clases de equivalencia).

3.2. PRUEBAS DE CAJA NEGRA

Estas pruebas ayudan a encontrar errores en las siguientes categorías:

- ▶ Funciones incorrectas o ausentes.
- ▶ Errores de interfaz.
- ▶ Errores en estructuras de datos o en acceso a base de datos externas.
- ▶ Errores de rendimiento.
- ▶ Errores de inicialización y de terminación.

3.2.1. CLASES DE EQUIVALENCIA (PARTICIÓN)

El problema que tienen las pruebas de Caja Negra es que los rangos de datos de entrada son muy amplios y sería muy costoso probar todos los valores de un rango, para evitar realizar esto se agrupan los datos en lo que llamamos Clases de Equivalencia. Se trata de agrupar los valores en clases equivalentes de forma que con probar un solo caso de cada clase, se prueban todos.

Este método divide los valores de los campos de entrada de un programa en clases de equivalencia, válidas y no válidas.

Ejemplo: para un campo de entrada llamado número de empleado definido como un numérico de 3 dígitos no pudiendo ser el primero 0 se definiría la siguientes clases de equivalencia:

- ▶ Clase válida: número comprendido entre 100 y 999.
- ▶ Clase no válida: número menor que 100.

3.2.1. CLASES DE EQUIVALENCIA (PARTICIÓN)

Para crear las clases de equivalencia existen unas normas:

- ▶ **Rango:** Si un parámetro de entrada debe estar comprendido en un cierto rango, aparecen 3 clases de equivalencia: por debajo (no válida), en el propio rango (válida) y por encima del rango (no válida).
- ▶ **Valor específico:** Si una entrada requiere un valor concreto, aparecen 3 clases de equivalencia: la del valor en cuestión (válida) y otras dos con dos valores diferentes al específico (no válidas).
- ▶ **Miembro de un conjunto:** Si una entrada requiere un valor de entre los de un conjunto, aparecen 2 clases de equivalencia: en el conjunto (válida: con tantas subclases como valores conocidos) y fuera del conjunto (no válida: cualquier otro valor).

Si se sabe de antemano que cada valor válido tiene un tratamiento diferente, se ha de crear una clase por cada uno de esos valores válidos y otra para uno que no lo sea.
- ▶ **Lógica:** Si una entrada es booleana, hay 2 clases: si (válida) o no (no válida).

3.2.1. CLASES DE EQUIVALENCIA (PARTICIÓN)

Condiciones de entrada	Clases válidas	Clases no válidas
Rango	Una clase que contemple los valores del rango.	Una clase con un valor por encima del rango. Una clase con un valor por debajo del rango.
Valor específico	Una clase que contemple ese valor.	Una clase con un valor por encima. Una clase con un valor por debajo.
Miembro de un conjunto	Una clase por cada uno de los miembros del conjunto.	Una clase con un valor que no pertenece al conjunto.
Lógica	Una clase que cumpla la condición	Una clase que no cumpla la condición

3.2.1. CLASES DE EQUIVALENCIA (PARTICIÓN)

Los mismos criterios se aplican a las salidas esperadas: hay que intentar generar resultados en todas y cada una de las clases.

Para representar las clases de equivalencia se usa una tabla, donde las filas serán las condiciones de entrada. Las columnas serán las clases de equivalencia que se generan según el tipo de la condición de entrada. Se asignará un código a cada una de las clases para mayor usabilidad.

Posteriormente, generaremos otra tabla donde se mostrarán los casos de prueba que se hayan obtenido combinando las diferentes clases. *El objetivo es probar todas las clases válidas y no válidas al menos una vez cada una de ellas.*

La tabla se rellena con todas las posibles combinaciones de las clases válidas con las no válidas. En un caso de prueba pueden aparecer varias clases de entrada válidas. No se deben generar casos de prueba con más de una clase no válida. Estos casos seguramente serían casos que probablemente no se ejecutarían nunca, ya que la primera decisión no válida evita que el programa siga ejecutando y se pueda llegar a la siguiente.

3.2.1. CLASES DE EQUIVALENCIA (PARTICIÓN)

EJEMPLO1. Dados los datos de un Empleado, el sistema ha de asignarle un salario, previa comprobación de que los datos son correctos.

- ▶ Los parámetros de entrada son:
 - ▶ Código: número de 3 dígitos que no empieza ni por 0.
 - ▶ Departamento: número de 2 dígitos (puede estar en blanco).
 - ▶ Puesto: "Analista", "Programador", "Diseñador"
- ▶ Si las entradas son correctas, entonces se les asignarán a los empleados los siguientes salarios:
 - ▶ Si el puesto es Analista → 2500 (salida S1)
 - ▶ Si el puesto es Diseñador → 1500 (salida S2)
 - ▶ Si el puesto es Programador → 2000 (salida S3)
- ▶ Si hay alguna entrada mal, el sistema ha de devolver los siguientes mensajes:
 - ▶ Si el Código no es correcto → salida Error1 (E1)
 - ▶ Si el Departamento no es correcto → salida Error2 (E2)
 - ▶ Si no se ha seleccionado Puesto → salida Error (E3)

3.2.1. CLASES DE EQUIVALENCIA (PARTICIÓN)

Con toda esta información, podemos generar la siguiente tabla con las clases de equivalencia:

Condición de entrada	Tipo de condición	Clases válidas (CVs)	Códigos de CVs	Clases no válidas (CNVs)	Códigos de CNVs
Código	Rango	100 <= Código <= 999	V1	Código < 100	NV1
				Código > 999	NV2
Departamento	Lógica	Está vacío	V2	No es un número	NV3
	Valor	Número de dos dígitos	V3	Nº de más de 2 dígitos	NV4
				Nº de menos de 2 dígitos	NV5
Puesto	Miembro de un conjunto	Puesto = "Programador"	V4	Puesto = "Elige puesto"	NV6
		Puesto = "Analista"	V5		
		Puesto = "Diseñador"	V6		

3.2.1. CLASES DE EQUIVALENCIA (PARTICIÓN)

Una vez definidas las claves, pasamos a generar los casos de prueba:

Caso de prueba	Clases de equivalencia	Condiciones de entrada			Resultado esperado
		Código	Departamento	Puesto	
CP01	V1, V2, V4	200		Programador	S3
CP02	V1, V2, V5	400		Analista	S1
CP03	V1, V2, V6	300		Diseñador	S2
CP04	V1, V3, V4	150	01	Programador	S3
CP05	V1, V3, V5	225	10	Analista	S1
CP06	V1, V3, V6	180	99	Diseñador	S2
CP07	NV1, V2, V4	99		Programador	E1
CP08	NV1, V2, V5	88		Analista	E1
CP09	NV1, V2, V6	77		Diseñador	E1
CP10	NV2, V2, V4	1125		Programador	E1
CP11	NV2, V2, V5	3698		Analista	E1
CP12	NV2, V2, V6	5254		Diseñador	E1
CP13	V1, NV3, V4	210	AD	Programador	E2
CP14	V1, NV3, V5	562	FG	Analista	E2
CP15	V1, NV3, V6	853	EH	Diseñador	E2
CP16	V1, NV4, V4	250	333	Programador	E2
CP17	V1, NV4, V5	547	555	Analista	E2
CP18	V1, NV4, V6	145	888	Diseñador	E2
CP19	V1, NV5, V4	542	5	Programador	E2
CP20	V1, NV5, V5	562	2	Analista	E2
CP21	V1, NV5, V6	214	4	Diseñador	E2
CP22	V1, V2, NV6	500		Elige puesto	E3
CP23	V1, V3, NV6	300	20	Elige puesto	E3
...	...	...	...	...	...

3.2.1. CLASES DE EQUIVALENCIA (PARTICIÓN)

EJEMPLO2. Se dispone del siguiente programa en Java. Recibe como parámetro d e entrada un número entero y la salida será la cadena “Par” si el número introducido es par e “Impar” si el número introducido es impar:

```
public String parImpar(int numero) {  
    String rdo = "";  
    if (numero % 2 == 0)  
        rdo = "Par";  
    else  
        rdo = "Impar";  
    return rdo;  
}
```

Se trata de diseñar el juego de pruebas de caja negra de este programa usando el método de partición o clase de equivalencia.

3.2.1. CLASES DE EQUIVALENCIA (PARTICIÓN)

Entonces, tendremos que la entrada es un número entero, es decir que pertenece al conjunto de los enteros. Entonces se generarían dos clases:

- ▶ Válida: Es un número entero
- ▶ No válida: No es un número entero (puede ser otro tipo de número o una cadena, ...)

Condición de entrada	Tipo de condición	Clases válidas (CVs)	Códigos de CVs	Clases no válidas (CNVs)	Códigos de CNVs
Numero	Miembro de un conjunto	Numero entero	V1	Número no entero	NV1
				Otra entrada	NV2

Además, nos damos cuenta de que el tratamiento de los números enteros se realiza de forma diferente si es par o impar, de modo que podemos subdividir nuestra clase válida en dos. Entonces la tabla quedaría:

Condición de entrada	Tipo de condición	Clases válidas (CVs)	Códigos de CVs	Clases no válidas (CNVs)	Códigos de CNVs
Numero	Miembro de un conjunto	Numero entero PAR	V1	Número no entero	NV1
		Numero entero IMPAR	V2	Otra entrada	NV2

3.2.1. CLASES DE EQUIVALENCIA (PARTICIÓN)

Las posibles salidas son:

Par (S1)

Impar (S2)

ERROR1 (E1)

ERROR2 (E2)

Entonces, nuestros casos de prueba serían:

Caso de prueba	Clases de equivalencia	Condiciones de entrada	Resultado esperado
		numero	
CP01	V1	20	S1
CP02	V2	-157	S2
CP03	NV1	7.47	E1
CP04	NV2	"hola"	E2

3.2.1.1. ANÁLISIS DE VALORES LÍMITE

Esta técnica es complementaria a las Clases de Equivalencia y explora las condiciones límites de un programa.

Es más probable que una función falle cuando los valores de entrada a la misma se sitúan en el límite de valores del rango de la función. La estadística así lo demuestra, aunque la causa formal no está clara.

Están pensadas para probar los límites en los siguientes casos:

- ▶ Para rangos de valores, se deben escoger casos por encima y por debajo de los límites superior e inferior, los propios valores de los límites y algunos valores intermedios para completar las pruebas.
- ▶ Si las estructuras internas (p.ej. arrays, ...) tienen límites, hay que generar casos que prueben el primer y último de los elementos.
- ▶ Si una condición de entrada especifica un número de valores, por ejemplo, si el programa requiere leer de 2 a 10 números de un fichero, habrá que probar con ficheros de 0, 1, 9, 10 y 11 elementos. Debemos verificar y definir el comportamiento del programa para cada uno de estos casos.

3.2.1.1. ANÁLISIS DE VALORES LÍMITE

EJEMPLO 1. Si una variable toma valores enteros entre 20 y 500 (rango de valores) se deben escoger los siguientes casos de prueba:

- ▶ 19 >> Justo debajo del rango
- ▶ 20 >> El límite inferior del rango
- ▶ 21 >> Dentro del rango
- ▶ Un valor cualquiera de la mitad del rango (entre 22 y 498)
- ▶ 499 >> Valor debajo del límite superior
- ▶ 500 >> El límite superior
- ▶ 501 >> Justo por encima del rango superior

3.2.1.1. ANÁLISIS DE VALORES LÍMITE

EJEMPLO 2. Si recordamos el ejemplo 1 de la clases de equivalencia, teníamos el parámetro de entrada **Código del empleado**, un número de 3 dígitos que no empieza por 0.

Sería un rango de valores: $100 \leq \text{Código} \leq 999$:

Podríamos generar los siguientes casos de prueba con relación a los valores límite para ese parámetro de entrada:

Caso de prueba	Clases de equivalencia	Condiciones de entrada			Resultado esperado
		Código	Departamento	Puesto	
CP50	V1, V3, V4	100	20	Programador	S3
CP51	V1, V2, V5	999		Analista	S1
CP52	NV1, V3, V6	99	30	Diseñador	ERR1
CP53	NV1, V2, V4	1000		Programador	ERR1

4. PRUEBAS DE INTEGRACIÓN

4. PRUEBAS DE INTEGRACIÓN

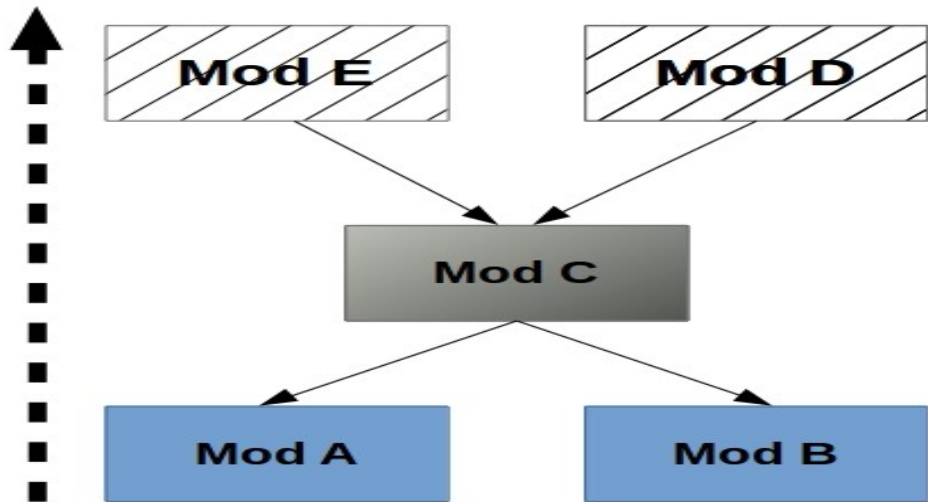
Son aquellas pruebas que comprueban que los componentes probados individualmente funcionan correctamente trabajando juntos, es decir, integrados. Se realizan después de las pruebas de unidad.

En el caso de la programación estructurada, estas pruebas se realizan de forma gradual y se pueden realizar de dos formas:

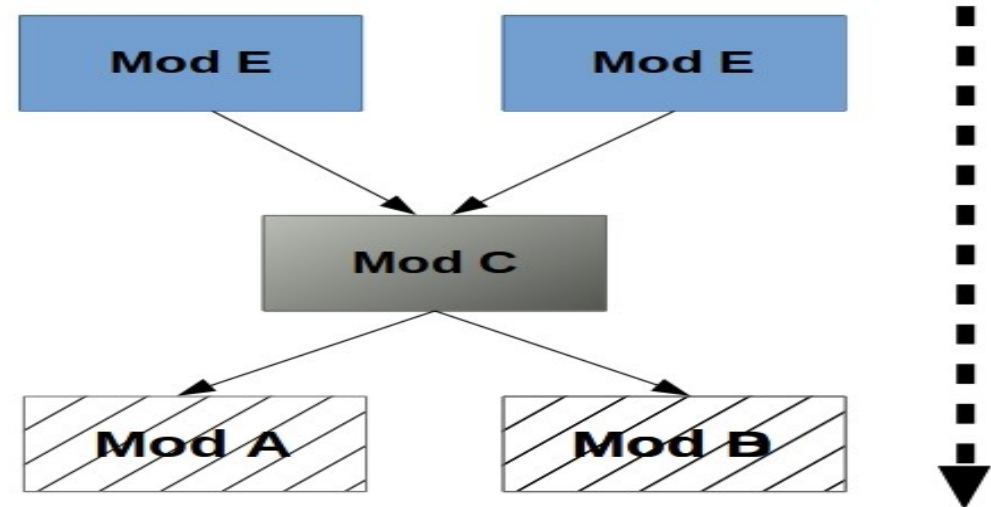
- ▶ **Incremental.** El software se va probando a medida que se va construyendo. En cada paso, se combina el siguiente módulo que se quiere probar con el conjunto de módulos que ya están probados. Puede ser de dos tipos:
 - ▶ Ascendente: la prueba comienza desde los niveles más bajos del sistema.
 - ▶ Descendente: se comienza probando el módulo principal, y a partir de ahí se van probando los módulos inferiores.
- ▶ **No incremental** (o big bang). Se prueba cada módulo por separado, y luego se integran todos a la vez y se prueba el sistema completo. Suelen salir muchos errores y la corrección se convierte en un proceso arduo y complicado.

4. PRUEBAS DE INTEGRACIÓN

Integración **ascendente**



Integración **descendente**



Ya probado



En pruebas



Sin probar o simulado

4. PRUEBAS DE INTEGRACIÓN

En el caso de **la programación orientada a objetos**, el objetivo de estas pruebas, es comprobar si las clases de las que consta el software funcionan correctamente cuando cooperan entre ellas con el objetivo de realizar las tareas encomendadas al programa.

Existen dos estrategias para las pruebas de integración en los sistemas orientados a objetos:

- ▶ Prueba basada en **hebra**: integra el conjunto de clases requeridas para responder a una entrada o evento del sistema. Cada hebra se integra y prueba de manera individual.
- ▶ Prueba basada en **uso**: se comienza probando las clases independientes, esto es, aquellas que usan muy pocas clases de servidor o ninguna. Después de probar estas, se examina la siguiente capa de clases, llamadas clases dependientes, que son utilizadas por las primeras. Esta secuencia de pruebas para las capas de clases dependientes continúa hasta que se prueba todo el sistema.

5. PRUEBAS DE VALIDACIÓN

5. PRUEBAS DE VALIDACIÓN

El objetivo de la prueba de validación (o de aceptación), es determinar si el software es considerado válido por parte la persona usuaria y si está preparado para su implantación en el entorno en el que se vaya a usar.

Estas pruebas las realiza el **cliente**. Son básicamente pruebas funcionales de caja negra, sobre el sistema completo, y buscan una cobertura de la especificación de requisitos y del manual del usuario. Estas pruebas no se realizan durante el desarrollo, debido a que el programa en desarrollo no es presentable al cliente, sino una vez pasadas todas las pruebas de integración por parte del desarrollador.

Los más expertos pueden demostrar que después de que un sistema supere todas las pruebas realizadas por el desarrollador, todavía aparecen una serie de errores que sólo aparecen cuando el cliente utiliza el sistema.

5. PRUEBAS DE VALIDACIÓN

Aquí se presenta un grave problema, ya que una vez finalizada totalmente la aplicación se presentan una serie de errores que debe arreglar, para evitar poner en marcha una aplicación que no es correcta y para corregir todo lo posible antes de una vez distribuida, ya que entonces la corrección de los es más complejo. Se practican dos tipos de pruebas, las llamadas Pruebas Alfa y Pruebas Beta.

Este tipo de pruebas, alfa y beta, son habituales en productos que se van a vender a muchos clientes. Algunos de los potenciales compradores se prestan a estas pruebas bien por ir entrenando a su personal con tiempo, bien a cambio de alguna ventaja económica (mejor precio sobre el producto final, derecho a mantenimiento gratuito, a nuevas versiones, etcétera). La experiencia muestra que estas prácticas son muy eficaces. Y de ahí vienen los famosos programas Beta.

5. PRUEBAS DE VALIDACIÓN

Pruebas Alfa

Las pruebas alfa consisten en invitar al cliente a que venga al entorno de desarrollo a probar el sistema. Se trabaja en un entorno controlado y el cliente siempre tiene un miembro del equipo de desarrollo a mano para ayudarlo a usar el sistema y para anotar y analizar los resultados de las pruebas.

Pruebas Beta

Las pruebas beta se realizan después de las pruebas alfa, y se desarrollan en el entorno del cliente, un entorno que está fuera de control. Aquí el cliente se queda a solas con el producto y trata de encontrarle fallos (reales o imaginarios) de los que informa al desarrollador.

6. PRUEBAS DE SISTEMA

6. PRUEBAS DE SISTEMA

Consiste en el proceso de prueba de un sistema integrado de hardware y software para comprobar si cumple con los requisitos especificados en el análisis, que son:

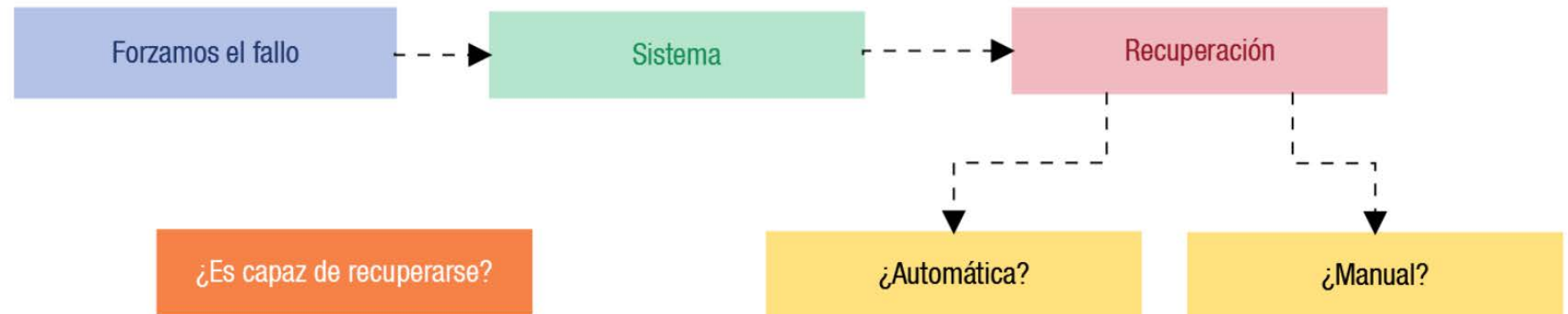
- ▶ Cumplimiento de todos los requisitos funcionales, considerando el producto software final al completo, en un entorno de sistema.
- ▶ El funcionamiento y rendimiento de las interfaces: hardware y software.
- ▶ Adecuación de la documentación de usuario.
- ▶ Ejecución y rendimiento en condiciones límite y de sobrecarga.

A continuación veremos los diferentes tipos.

6. PRUEBAS DE SISTEMA

Pruebas de recuperación.

En este tipo de pruebas, se fuerza un fallo del sistema y se verifica que se recupera de forma adecuada. Son especialmente importantes en los sistemas que deben ser tolerantes a fallos y en los que la recuperación debe realizarse de forma rápida.



6. PRUEBAS DE SISTEMA

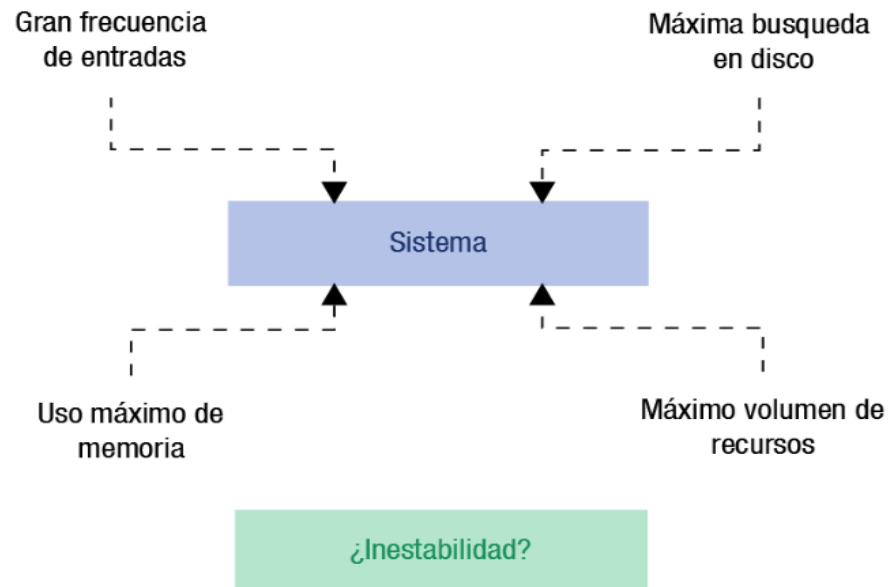
Prueba de seguridad.

Con este tipo de pruebas se trata de verificar que el sistema está protegido frente a accesos no lícitos u otro tipo de ataques. El objetivo es comprobar el comportamiento del sistema ante estas situaciones y valorar en qué medida la dificultad que supone el intento de penetración en el sistema no compensa los beneficios que pueda conseguir la persona que lo intenta penetrar. Son especialmente importantes en aquellos sistemas que manejan información sensible (económica, de salud, ...)



6. PRUEBAS DE SISTEMA

Pruebas de resistencia o esfuerzo. Son pruebas que ponen al sistema en situaciones que demandan gran cantidad de recursos. Se trata de exponer al sistema a situaciones extremas que, aun no siendo las circunstancias habituales, pueden ocurrir en determinado momento y comprobar su comportamiento ante ellas. Ejemplos de este tipo de pruebas son: ejecutar casos de prueba que requieran memoria máxima y otros recursos, que provoquen una búsqueda excesiva de datos residentes en disco, etc ...



7. DOCUMENTACIÓN DE LAS PRUEBAS

7. DOCUMENTACIÓN DE LAS PRUEBAS

El standard IEEE 829-1998 (<https://standards.ieee.org/standard/829-1998.html>) define cómo han de documentarse las pruebas realizadas sobre un software.

Durante esta fase se genera mucha documentación, formal e informal. Hay varios documentos importantes donde se recogen de manera formal el diseño y los resultados de las pruebas:

Plan de pruebas.

Como su nombre indica recoge el plan o la estrategia de pruebas. El enfoque, el alcance, los recursos y el calendario. Se suele realizar un plan maestro, y otros planes dentro de cada uno de los niveles del sistema. Detalla:

- ▶ Los elementos que se van a probar y qué característica específica de ellos se está probando.
- ▶ Las tareas y los responsables de las tareas.
- ▶ Los riesgos asociados al plan.

7. DOCUMENTACIÓN DE LAS PRUEBAS

Especificación de prueba.

Cada prueba está definida en tres documentos diferentes:

- ▶ La especificación del diseño de prueba. Donde se definen los requisitos incluidos en la prueba, los casos o escenarios de prueba, los procedimientos asociados y la definición del criterio de si el resultado de la prueba es válido o no.
- ▶ Especificación de los casos de prueba. Documenta los valores introducidos en los diferentes casos de prueba, las salidas previstas y las salidas obtenidas.
- ▶ La especificación de los procedimientos de prueba. Detalla los pasos a seguir para hacer funcionar el sistema y ejecutar los casos de prueba definidos en los otros documentos. Detalla cómo ejecutar cada prueba, incluidas las condiciones previas de configuración y los pasos que deben seguirse.

7. DOCUMENTACIÓN DE LAS PRUEBAS

Informes de pruebas.

Son cuatro tipos de documentos:

- ▶ Informe de los elementos que están siendo probados.
- ▶ Registro de pruebas. Es un registro cronológico sobre la ejecución de las pruebas (qué casos de prueba se ejecutaron, quién los ejecutó, en qué orden y si cada prueba pasó o no).
- ▶ Informe de incidentes o anomalías (donde se anota cualquier detalle que surja durante las pruebas que requiera ser investigado). Una anomalía no es necesariamente un error, puede ser un problema, un incidente ocurrido durante la prueba, un defecto, ...
- ▶ Informe resumen de las actividades de prueba. Sirve para resumir los resultados de los niveles de las actividades de prueba designadas y proporcionar evaluaciones basadas en estos resultados.

8. PLANIFICACIÓN DE LAS PRUEBAS

8. PLANIFICACIÓN DE LAS PRUEBAS

La Fase de Pruebas, por su envergadura e importancia, necesita una organización seria y fiable. Ante una fase de pruebas, se debe tomar como axioma que se van a encontrar errores.

Los componentes de una planificación serán:

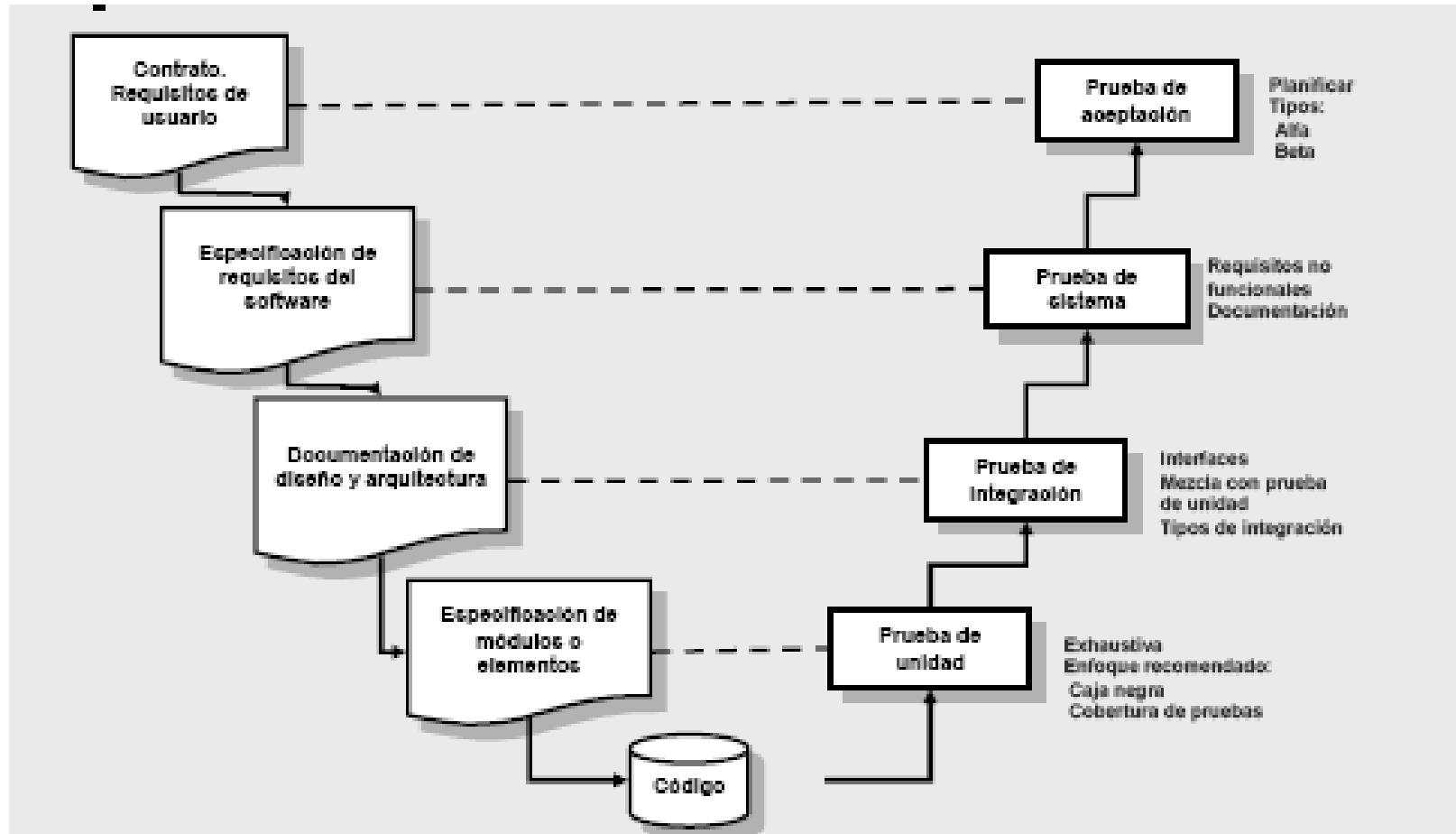
1. **Objetivos:** Definir los objetivos de cada fase de las pruebas.
2. **Criterios de terminación:** Especificar cuando se deben acabar las pruebas.
3. **Cronología:** Fijar los tiempos necesarios para cada fase (diseño, escritura y ejecución)
4. **Responsabilidades:** Especificar los responsables de cada fase, así como quién corregirá los errores detectados.
5. **Bibliotecas de casos de prueba y normas:** Crear una sistemática de identificación, escritura y almacenamiento de casos de prueba.

8. PLANIFICACIÓN DE LAS PRUEBAS

6. Herramientas: Establecer cuáles serán las herramientas de prueba que se van a utilizar.
7. Tiempo de máquina: Determinar el tiempo de computación que se necesita en cada fase del proyecto de prueba.
8. Configuración del equipo: Detallar la necesidad de configuraciones especiales de equipo o de algún período concreto.
9. Integración: Describir el plan de integración del sistema.
10. Métodos de seguimiento: Especificar los métodos que se han de utilizar en las pruebas.
11. Depuración: Definir un mecanismo para informar sobre los errores detectados, para seguir el proceso de las correcciones y para incorporar éstas al sistema.

8. PLANIFICACIÓN DE LAS PRUEBAS

El orden de realización de las pruebas podría ser:



8. PLANIFICACIÓN DE LAS PRUEBAS

En cuanto a las pruebas unitarias un orden de estas podría ser:

1. Pasar pruebas de caja negra analizando valores límite (recordamos que hay que analizar condiciones límite de entrada y de salida).
2. Identificar clases de equivalencia de datos (entrada y salida) y añadir más pruebas de caja negra para contemplar valores normales (en las clases de equivalencia en que estos sean diferentes de los valores límite; es decir, en rangos amplios de valores).
3. Añadir pruebas basadas en "presunción de error". A partir de la experiencia y el sentido común, se preparan situaciones que pueden provocar error, y se buscan errores en esos puntos.
4. Medir la cobertura de caja blanca que se ha logrado con las fases previas y añadir más pruebas de caja blanca hasta lograr la cobertura deseada.

8. PLANIFICACIÓN DE LAS PRUEBAS: MÉTRICA v3

Métrica es una metodología para el desarrollo y mantenimiento de Sistemas de Información que fue desarrollada por el Ministerio de Hacienda y Administraciones Públicas de España. Con esta herramienta se logra dar soporte a grandes sistemas de información en el ciclo de vida del software además de proporcionar un marco de gestión que asegure que los proyectos cumplen sus objetivos en términos de calidad, coste y plazos.

Los objetivos que se quieren conseguir con esta metodología son:

- ▶ Dotar a la organización de productos software.
- ▶ Mejorar la productividad de los departamentos de sistemas y tecnologías de la información y las comunicaciones.
- ▶ Facilitar la operación, mantenimiento y uso de productos software.
- ▶ Definir sistemas de información que ayuden a conseguir los fines de la organización.

8. PLANIFICACIÓN DE LAS PRUEBAS: MÉTRICA v3

Las fases que engloba esta metodología al utilizarla en la gestión de un Sistema Información son:

- ▶ Planificación del SI.
- ▶ Desarrollo del SI. Este a su vez engloba:
 - ▶ Estudio de viabilidad del sistema (EVS).
 - ▶ Análisis del sistema de información (ASI).
 - ▶ Diseño del sistema de información (DSI).
 - ▶ Construcción del sistema de información (CSI).
 - ▶ Implantación y aceptación del sistema (IAS).
- ▶ Mantenimiento del SI.

8. PLANIFICACIÓN DE LAS PRUEBAS: MÉTRICA v3

Según Métrica, el plan de pruebas:

- ▶ Sirve como guía para la realización de las pruebas.
- ▶ Permite verificar que el sistema de información cumple las necesidades establecidas por el usuario.
- ▶ Define los objetivos de la prueba de un sistema establece y coordina una estrategia de trabajo, y provee del marco adecuado para elaborar una planificación paso a paso de las actividades de prueba. Estas actividades se incluyen en CSI y IAS.

Para saber más: <https://manuel.cillero.es/doc/metodologia/metrica-3/>

FUENTES

- ▶ Ramos Martín, Alicia y María Jesús, “Entornos de Desarrollo”, Ed. Garceta.
- ▶ Comesaña, José Luis, “Apuntes de Entornos de Desarrollo”